VOLUME 05 / 08

BUILD DOT4

Specify, encode and test a signed packed instruction

Implement a custom four-lane signed dot product and design a fair software/hardware comparison.

FIG 09 / DOT4 DATAFLOW

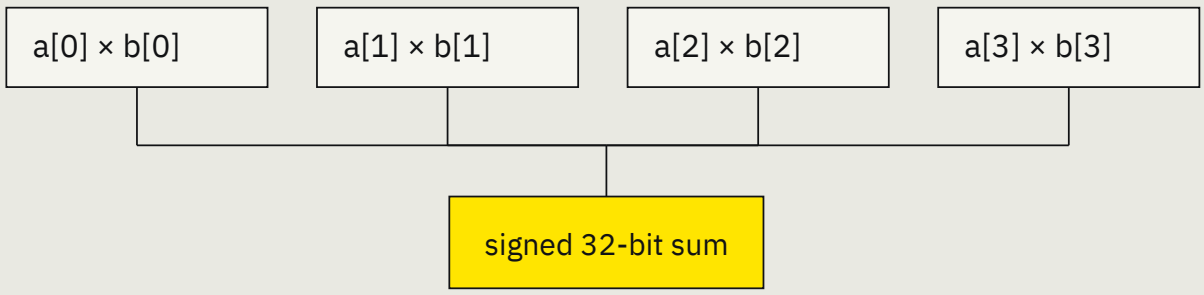


Illustration: independent signed-byte products feed one sum.

A hands-on companion to the RISC-V project. Examples, implementation steps, debugging and completion checks. Personal design edition / October 2026.

What you will build

You will write an exact custom-instruction contract, implement signed arithmetic in EX, encode it in tests and evaluate its limits.

Before you start

Understand operand forwarding and load interlocks. Know two’s-complement bytes.

Files and outputs

rtl/rv32_core.sv: signed dot loop and opcode 0x0b case. tests/run.py: r(), signed(), reference() and DOT4 cases.

Build order

1. Freeze lane semantics and result width.
2. Choose one exact custom encoding.
3. Implement signed products and a wide sum.
4. Extend the reference and directed tests.
5. Exercise dependencies and invalid encodings.
6. Plan a matched scalar comparison.

The checked-in core is an assistant-generated reference. Work on a learning branch. The code excerpts explain selected regions and are not complete replacement modules unless explicitly labeled.

DOT4 multiplies four signed byte pairs

The custom instruction treats each source register as four independent signed 8-bit lanes. Lane zero is the least significant byte. It multiplies corresponding lanes and sums all four products into a 32-bit result.

```
DOT4 rd, rs1, rs2
rd = sum(signed8(rs1.byte[i]) * signed8(rs2.byte[i]))
     for i = 0, 1, 2, 3
```

There is no accumulator input, rounding or saturation. The sum ranges from −65024 to 65536. A signed 16-bit result is too narrow. A signed 18-bit value can represent the full range, and the destination remains 32 bits.

The encoding uses custom-0: opcode `0x0B`, funct3=0 and funct7=0. Other function combinations trap. GNU assembly can emit `.insn r 0x0b, 0, 0, a0, a1, a2`.

This project-specific instruction illustrates a small hardware/software tradeoff. It is not a ratified vector extension or evidence of a lab's accelerator architecture.

FIG 09 / DOT4 DATAFLOW

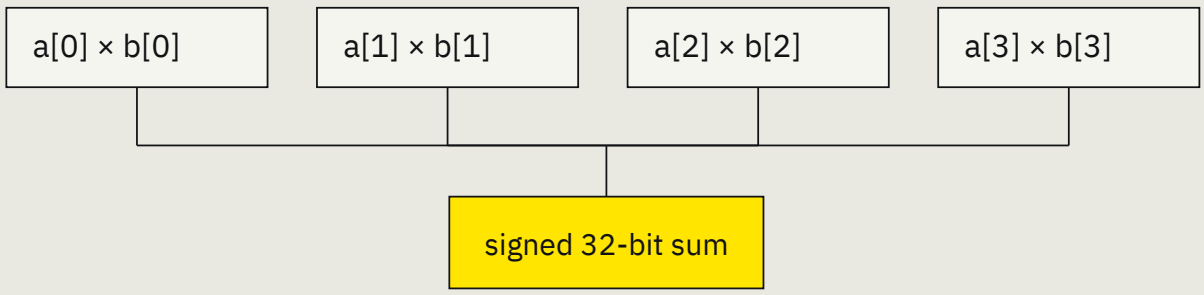


Illustration: independent signed-byte products feed one sum.

Lane interpretation changes the answer

For illustrative source words `A=0x0102FF80` and `B=0x020304FF`, decode each byte as signed before multiplying.

Lane	A byte	A signed	B signed	Product
0	0x80	−128	−1	128
1	0xFF	−1	4	−4
2	0x02	2	3	6
3	0x01	1	2	2
Total				132

The sum is `128 - 4 + 6 + 2 = 132`. The byte 0x80 means −128 here, and 0xFF means −1. Reading the whole word as one signed integer does not implement lane-wise arithmetic.

Four lanes of −128 multiplied by −128 produce `4 × 16384 = 65536`. Four lanes of −128 multiplied by 127 produce `4 × (-16256) = -65024`.

Tests should include isolated lanes, mixed signs, extrema, x0 destinations and dependencies. Directed and random programs cover packed sums. `make dot4-sweep` runs all 262,144 isolated lane pairs through the RTL.

Step 1: freeze the arithmetic contract

Write the operation before writing RTL. Lane i is bits 8i+7 through 8i of each source. Interpret each lane as signed independently, multiply corresponding values, and sum four products.

Choose explicitly whether the instruction accumulates old rd, saturates or rounds. This instruction does none of those. Its only input registers are rs1 and rs2; rd receives a signed 32-bit result.

The most positive product is $-128 \times -128 = 16384$. The most negative is $-128 \times 127 = -16256$. Four products therefore span −65024 through 65536. Select an intermediate width that can hold both extremes.

Write independent hand cases for all-zero lanes, one active lane, mixed signs, the maximum and the minimum. Keep a lane-order example with distinct bytes to reveal a swapped lane mapping.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 5A. What goes wrong if all four products are added into a signed 16-bit accumulator?

Step 2: allocate an exact encoding

Use the custom-0 major opcode reserved for project extensions. This implementation selects opcode 0x0b, funct3=0 and funct7=0, with normal R-format register fields.

```
// Inside the EX opcode case.
7'h0b: begin
    wr = 1;
    result = dot;
    legal = (funct3 == 0 && funct7 == 0);
end
```

Reject other custom-0 function combinations. Reserved custom space does not mean every possible function field is this instruction.

The local encoder is `r(0, rd, rs1, rs2, op=0x0b)`. Standard GNU assembly can emit the same R-format fields with `.insn r 0x0b, 0, 0, a0, a1, a2`. The directive emits bytes; it does not teach a stock simulator or compiler the operation's meaning.

No compiler backend change is needed for this first prototype. A custom mnemonic and scheduling support are separate later projects.

Step 3: preserve signedness and width in RTL

The loop operates on the already forwarded EX inputs a and b. Start the sum at zero in the combinational block.

```
reg signed [31:0] dot;
integer lane;
// Inside the combinational EX block.
dot = 0;
for (lane = 0; lane < 4; lane = lane + 1)
    dot = dot + ($signed(a[lane*8 +: 8]) *
                 $signed(b[lane*8 +: 8]));
```

The indexed part-select has a constant width of eight. The casts interpret each selected byte as signed. Keep the destination wide enough and verify how the simulator sizes the expression.

Test extrema and mixed signs immediately. An unsigned multiplication error can pass positive-only tests. A narrow sum can pass most random small inputs.

Synthesis will unroll the constant four-iteration loop into hardware. It does not imply four execution cycles. Here the combinational unit shares one EX interval.

Step 4: write a separate scalar oracle

The reference must unpack bytes and interpret them explicitly. Python's integer type does not automatically have signed 8-bit lane behavior.

```
def signed8(n):
    n &= 255
    return n - 256 if n & 128 else n

def dot4(a, b):
    return sum(signed8(a >> (8*i)) *
               signed8(b >> (8*i)) for i in range(4))
```

Check `dot4(0x0102ff80, 0x020304ff)`═`132`. Check the positive extreme 65536 and negative extreme −65024. Mask the architectural output to 32 bits when comparing RTL register values.

The local model already handles custom opcode 0x0b. When implementing your own version, update the checker from the written contract rather than translating the RTL line by line.

Run `dot_mixed` and `dot_extremes` in the teaching labs. An exhaustive isolated-lane operand sweep is additional work, not part of the reference claim.

Step 5: exercise real pipeline consumers

A correct combinational dot function can still fail as an instruction. Use an ALU producer feeding each source, a load feeding either source, DOT4 feeding another DOT4, and DOT4 feeding a store or branch.

The `dot_load` teaching case loads packed bytes and immediately uses them in DOT4. It should receive the same load interlock as ordinary two-source arithmetic. The subsequent store must receive the dot result through normal forwarding.

Try rd=x0 and invalid funct3/funct7. x0 must remain zero, and the invalid instruction must trap without a younger memory side effect.

Inspect `uses_rs1` and `uses_rs2`. Both must recognize the custom opcode. A decoder change without the matching hazard-source update creates a subtle load-use bug.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 5B. Why is adding only a new ALU case insufficient to integrate DOT4 safely?

Step 6: evaluate a fair comparison

Define identical input arrays, signed semantics and checked outputs for a scalar kernel and a DOT4 kernel. Count useful dot-product outputs, not only retired instructions.

Base RV32I has no MUL. The scalar baseline needs a documented software multiplication routine. An RV32IM binary changes the baseline. Include loads, packing and loop overhead if they belong to the intended workload.

Measure total cycles for both implementations. Route both configurations with the same shell, target, constraints, tools and seeds. Then use each configuration's own timing estimate to convert cycles to estimated execution time.

`make bench ablation` does this for the reference: 5.3× over the strongest RV32I baseline at N=256, clock cost included. Instruction-count reduction alone does not prove application speedup. Timing and LUTs alone do not measure energy.

Checkpoint: state a falsifiable hypothesis, such as “DOT4 reduces total cycles on a fixed packed kernel, but may increase the critical path.” Record the evidence before reaching a conclusion.

A research experiment for this part

Write this entry before editing. It is a proposed experiment, not a result already obtained by the learner.

Question	Do signed lanes and their sum match the written contract?
Prediction	The mixed-sign example returns 132 and extrema stay within a signed 18-bit range.
Experiment	Run dot_mixed, dot_load and dot_extremes, checking hand calculations and dependencies.
Evidence	Lane table, source words, traces and the arithmetic contract.

What to record

Save source revision, tool versions, input words, seed, exact command, expected and observed behavior, raw artifact path and limitation. End with the decision that the observation supports.

Use `docs/EXPERIMENT_LOG.md`. If the result contradicts the prediction, keep the failure and explain the next test. Do not rewrite the prediction afterward to make the experiment appear successful.

Answers and completion check

5A. The maximum sum 65536 exceeds signed 16-bit range. The accumulator wraps and produces a wrong architectural result.

5B. Encoding, legality, reference semantics, both source-use predicates and hazard consumers must agree. Otherwise a load can feed stale DOT4 operands even when the arithmetic unit itself is correct.

Completion check

Explain the exact encoding and lane math. Pass signed extrema, mixed lanes, dependencies and invalid encodings. Keep workload and physical-performance claims pending until their separate experiments run.

Sources and next steps

These sources support the concepts in this volume. The current RTL and local execution contract remain the authority for implementation-specific behavior.

1. RISC-V ISA
2. GNU assembler
3. Project source

Detailed chapter-mapped reading is in `docs/textbook/SOURCES.md`. Research-interest context is in `docs/RESEARCH.md`. The original overview and its interactive figures remain companion material.

Keep the evidence boundary

Teaching labs are separate from the 252-program baseline. Reference coverage and timing numbers belong to the recorded source/tool configuration. No physical board execution, ISA certification or formal proof is implied. The DOT4 speedup covers one kernel.

Continue with Volume 6. Run the complete regression before treating a learning edit as a finished implementation.